17.1 系统提示词的构建

> **生成模型**：Claude Opus 4.6 (anthropic/claude-opus-4-6)
> **Token 消耗**：输入 ~450,000 tokens，输出 ~36,000 tokens（本章合计）

在第 7 章和第 8 章里，我们已经看过 PiAgent 运行时怎么跑 Agent 循环，以及模型选择和故障转移是怎么工作的。但在 Agent 向大语言模型发出第一条请求之前，还有一个关键步骤——**构建系统提示词（System Prompt）**。

系统提示词是 LLM 对话的"元指令"，告诉模型"你是谁、能做什么、应该遵循哪些规则"。OpenClaw 的系统提示词不是一段静态文本，而是由多个**动态组装的片段**拼接而成的。不同的会话类型（主 Agent vs 子 Agent）、不同的运行环境（沙箱 vs 宿主机）、不同的通道（Telegram vs Signal）都会影响最终生成的提示词内容。

本节剖析系统提示词的三层构建体系：基础模板、运行时参数、以及最终的提示词诊断报告。

17.1.1 基础提示词模板（`src/agents/system-prompt.ts`）

PromptMode：三种提示词模式

OpenClaw 定义了三种提示词模式，控制系统提示词的"详尽程度"：

```
// src/agents/system-prompt.ts
export type PromptMode = "full" | "minimal" | "none";
```

模式	用途	包含内容
`"full"`	主 Agent 会话	所有功能段落：工具、安全、技能、记忆、消息、语音等
`"minimal"`	子 Agent 会话	仅保留：工具列表、工作区、运行时信息
`"none"`	极简模式	仅返回一行身份声明

当模式为 `"none"` 时，函数直接返回一行字符串：

```
if (promptMode === "none") {
  return "You are a personal assistant running inside OpenClaw.";
}
```

> **衍生解释——System Prompt 与 User Message 的区别**
>
> 在大语言模型的 API 中，消息一般分为三种角色： `system` （系统）、 `user` （用户）、 `assistant` （助手）。System Prompt 是放在对话最开头的"隐藏指令"，用户通常看不到，但它对模型行为有最强的约束力。OpenClaw 的所有行为规范——从工具使用到安全限制——都编码在 System Prompt 中。

buildAgentSystemPrompt：核心构建函数

`buildAgentSystemPrompt` 是整个系统提示词的入口。它接受一个庞大的参数对象，涵盖了 OpenClaw 运行时几乎所有可能影响提示词的维度：

```
// src/agents/system-prompt.ts（简化参数列表）
export function buildAgentSystemPrompt(params: {
  workspaceDir: string;              // 工作区目录
  promptMode?: PromptMode;            // 提示词模式
  toolNames?: string[];               // 可用工具名列表
  toolSummaries?: Record<string, string>; // 工具描述
  skillsPrompt?: string;              // 技能 XML 片段
  contextFiles?: EmbeddedContextFile[];// 上下文文件
  extraSystemPrompt?: string;         // 额外提示（如子 Agent 上下文）
  ownerNumbers?: string[];            // 所有者号码列表
  userTimezone?: string;              // 用户时区
  runtimeInfo?: { ... };              // 运行时环境信息
  sandboxInfo?: { ... };              // 沙箱配置
  memoryCitationsMode?: MemoryCitationsMode; // 记忆引用模式
  // ...更多参数
}): string { ... }
```

函数内部按照**固定顺序**将各个功能段落组装成一个字符串数组，最后用 `\n` 连接。下面逐一分析每个段落。

段落一：Tooling（工具列表）

这是提示词的第一个核心段落，把所有可用工具以列表形式呈现给模型：

```ts
// src/agents/system-prompt.ts
const coreToolSummaries: Record<string, string> = {
  read: "Read file contents",
  write: "Create or overwrite files",
  edit: "Make precise edits to files",
  exec: "Run shell commands (pty available for TTY-required CLIs)",
  web_search: "Search the web (Brave API)",
  browser: "Control web browser",
  canvas: "Present/eval/snapshot the Canvas",
  cron: "Manage cron jobs and wake events",
  message: "Send messages and channel actions",
  sessions_spawn: "Spawn a sub-agent session",
  // ...更多工具
};
```

系统维护了一个**工具排列顺序表** `toolOrder`，确保常用工具（`read`、`write`、`edit`）始终排在前面。对于不在标准列表中的**外部工具**（如 MCP 提供的自定义工具），则按字母排序追加到末尾：

```ts
const enabledTools = toolOrder.filter((tool) => availableTools.has(tool));
const toolLines = enabledTools.map((tool) => {
  const summary = coreToolSummaries[tool] ?? externalToolSummaries.get(tool);
  const name = resolveToolName(tool);
  return summary ? `- ${name}: ${summary}` : `- ${name}`;
});
// 追加外部工具
for (const tool of extraTools.toSorted()) {
  const name = resolveToolName(tool);
  toolLines.push(summary ? `- ${name}: ${summary}` : `- ${name}`);
}
```

工具名是**大小写敏感**的。OpenClaw 通过 `resolveToolName` 函数保留调用方传入的原始大小写，同时在内部用小写进行去重和匹配。

段落二：Tool Call Style（工具调用风格）

这一小段指导模型何时应该"默默地"调用工具，何时需要向用户解释：

```
Default: do not narrate routine, low-risk tool calls (just call the tool).
Narrate only when it helps: multi-step work, complex/challenging problems,
sensitive actions (e.g., deletions), or when the user explicitly asks.
```

段落三：Safety（安全规范）

安全段落是 OpenClaw 遵循 Anthropic 宪法式 AI 原则的体现：

```
const safetySection = [
  "## Safety",
  "You have no independent goals: do not pursue self-preservation, " +
    "replication, resource acquisition, or power-seeking...",
  "Prioritize safety and human oversight over completion...",
  "Do not manipulate or persuade anyone to expand access or disable safeguards...",
];
```

> **衍生解释——Constitutional AI**
>
> Constitutional AI（宪法式 AI）是 Anthropic 提出的一种 AI 对齐方法。其核心思想是为 AI 制定一套明确的行为准则（"宪法"），并通过训练让模型内化这些准则。OpenClaw 的 Safety 段落就是这种理念的具体实施——通过系统提示词的方式将安全边界植入到每一次 Agent 对话中。

段落四：Skills（技能系统）

技能段落在"full"模式下才会出现（`isMinimal` 时跳过）。它指导模型如何从 `<available_skills>` 列表中选择合适的技能：

```
function buildSkillsSection(params: {
  skillsPrompt?: string;
  isMinimal: boolean;
  readToolName: string;
}) {
  if (params.isMinimal) return [];
  const trimmed = params.skillsPrompt?.trim();
  if (!trimmed) return [];
  return [
    "## Skills (mandatory)",
    "Before replying: scan <available_skills> <description> entries.",
    `- If exactly one skill clearly applies: read its SKILL.md at <location> ` +
      `with \`${params.readToolName}\`, then follow it.`,
    "- If multiple could apply: choose the most specific one...",
    "- If none clearly apply: do not read any SKILL.md.",
    trimmed,  // 实际的技能 XML 列表
  ];
}
```

技能 XML 片段的格式如下（由 Skills 模块生成，详见后续章节）：

```
<available_skills>
  <skill>
    <name>git-master</name>
    <description>MUST USE for ANY git operations...</description>
    <location>/path/to/SKILL.md</location>
  </skill>
</available_skills>
```

段落五：Memory Recall（记忆召回）

当工具列表中包含 `memory_search` 或 `memory_get` 时，记忆段落指导模型在回答涉及历史信息的问题前先搜索记忆文件：

```
function buildMemorySection(params: {
  isMinimal: boolean;
  availableTools: Set<string>;
  citationsMode?: MemoryCitationsMode;
}) {
  if (params.isMinimal) return [];
  if (!params.availableTools.has("memory_search") &&
      !params.availableTools.has("memory_get")) {
    return [];
  }
  const lines = [
    "## Memory Recall",
    "Before answering anything about prior work, decisions, dates, " +
      "people, preferences, or todos: run memory_search on MEMORY.md + " +
      "memory/*.md; then use memory_get to pull only the needed lines.",
  ];
  // 根据 citationsMode 决定是否要求引用来源
  if (params.citationsMode === "off") {
    lines.push("Citations are disabled...");
  } else {
    lines.push("Citations: include Source: <path#line>...");
  }
  return lines;
}
```

段落六至十：条件性段落

以下段落根据运行时条件决定是否出现：

段落	条件	功能
User Identity	配置了 ownerNumbers 且非 minimal	告知模型哪些号码是"主人"
Current Date & Time	配置了 userTimezone	提供当前时区信息
Reply Tags	非 minimal 模式	指导使用 `[[reply_to_current]]` 等回复标签
Messaging	非 minimal 且有 message 工具	跨通道消息发送指引
Voice (TTS)	配置了 TTS hint	语音合成行为指导

以 `buildMessagingSection` 为例，它的逻辑特别复杂，因为需要处理不同通道的消息能力：

```
function buildMessagingSection(params: {
  isMinimal: boolean;
  availableTools: Set<string>;
  messageChannelOptions: string;    // 如 "signal|telegram|discord"
  inlineButtonsEnabled: boolean;    // 当前通道是否支持内联按钮
  runtimeChannel?: string;          // 当前运行通道
}) {
  if (params.isMinimal) return [];
  return [
    "## Messaging",
    "- Reply in current session → automatically routes to the source channel",
    "- Cross-session messaging → use sessions_send(sessionKey, message)",
    params.availableTools.has("message")
      ? [
          "### message tool",
          "- Use `message` for proactive sends + channel actions...",
          `- If multiple channels, pass \`channel\` (${params.messageChannelOptions}).`
          params.inlineButtonsEnabled
            ? "- Inline buttons supported..."
            : `- Inline buttons not enabled for ${params.runtimeChannel}...`,
        ].join("\n")
      : "",
  ];
}
```

段落十一：Workspace（工作区）

工作区段落始终存在（即使在 minimal 模式下），告知模型当前的工作目录：

```
## Workspace
Your working directory is: /home/user/.openclaw/workspace
Treat this directory as the single global workspace for file operations.
```

段落十二：Sandbox（沙箱）

当启用沙箱运行时（Docker 隔离环境），会插入详细的沙箱信息：

```
params.sandboxInfo?.enabled ? [
  "## Sandbox",
  "You are running in a sandboxed runtime (tools execute in Docker).",
  params.sandboxInfo.workspaceDir
    ? `Sandbox workspace: ${params.sandboxInfo.workspaceDir}`
    : "",
  params.sandboxInfo.elevated?.allowed
    ? "Elevated exec is available for this session."
    : "",
  // ...更多沙箱细节
].filter(Boolean).join("\n") : ""
```

> **衍生解释——沙箱（Sandbox）**
>
> 沙箱是一种安全隔离机制，将程序的执行限制在一个受控环境中，防止其影响宿主系统。OpenClaw 通过 Docker 容器实现沙箱——Agent 的工具调用（如 `exec` 、 `write` ）在容器内执行，即使模型产生了危险命令（如 `rm -rf /` ），也不会损害宿主机。沙箱还支持"提权"（elevated）模式，允许在特定条件下执行宿主机命令。

段落十三：上下文文件注入

这是提示词中**体积最大**的部分。OpenClaw 将工作区中的引导文件（如 AGENTS.md、SOUL.md）以 `# Project Context` 标题注入：

```
const contextFiles = params.contextFiles ?? [];
if (contextFiles.length > 0) {
  const hasSoulFile = contextFiles.some((file) => {
    const baseName = file.path.split("/").pop() ?? file.path;
    return baseName.toLowerCase() === "soul.md";
  });
  lines.push("# Project Context",
    "The following project context files have been loaded:");
  if (hasSoulFile) {
    lines.push(
      "If SOUL.md is present, embody its persona and tone. " +
        "Avoid stiff, generic replies; follow its guidance..."
    );
  }
  for (const file of contextFiles) {
    lines.push(`## ${file.path}`, "", file.content, "");
  }
}
```

注意这里对 `SOUL.md` 文件做了特殊处理——如果存在 SOUL 文件，模型会被指示"体现其人格和语调"。这是 OpenClaw 身份系统的一部分（详见 9.3 节）。

段落十四至十六：静默回复、心跳、运行时

Silent Replies（静默回复）：当模型判断无需回复时，返回一个特殊 token（`SILENT_REPLY_TOKEN`），OpenClaw 据此判断是否需要向用户发送消息。

Heartbeats（心跳）：OpenClaw 定期向 Agent 发送心跳消息，如果一切正常，Agent 回复 `HEARTBEAT_OK`。这个机制用于检测 Agent 是否需要主动报告异常。

Runtime（运行时信息）：最后一行包含当前运行时的全部元数据，格式为：

```
Runtime: agent=default | host=macbook | repo=/project | os=Darwin 24.0 (arm64) |
  node=v22.0.0 | model=anthropic/claude-sonnet-4-20250514 | channel=telegram |
  capabilities=inlineButtons | thinking=off
```

这行信息由 `buildRuntimeLine` 函数生成：

```
export function buildRuntimeLine(
  runtimeInfo?: { ... },
  runtimeChannel?: string,
  runtimeCapabilities: string[] = [],
  defaultThinkLevel?: ThinkLevel,
): string {
  return `Runtime: ${[
    runtimeInfo?.agentId ? `agent=${runtimeInfo.agentId}` : "",
    runtimeInfo?.host ? `host=${runtimeInfo.host}` : "",
    runtimeInfo?.repoRoot ? `repo=${runtimeInfo.repoRoot}` : "",
    runtimeInfo?.model ? `model=${runtimeInfo.model}` : "",
    runtimeChannel ? `channel=${runtimeChannel}` : "",
    `thinking=${defaultThinkLevel ?? "off"}`,
  ].filter(Boolean).join(" | ")}`;
}
```

完整的提示词组装流程

把上述所有段落串联起来， `buildAgentSystemPrompt` 的完整流程如下：

```
┌──────────────────────────────────────────┐
│  "You are a personal assistant..."        │  身份声明
├──────────────────────────────────────────┤
│  ## Tooling                               │  工具列表（始终存在）
│  ## Tool Call Style                       │  调用风格
│  ## Safety                                │  安全规范
├──────────────────────────────────────────┤
│  ## OpenClaw CLI Quick Reference          │  CLI 参考
│  ## Skills (mandatory)         [full only]  │  技能列表
│  ## Memory Recall              [full + 工具]  │  记忆召回
│  ## OpenClaw Self-Update       [full + gw]  │  自更新
│  ## Model Aliases              [full only]  │  模型别名
├──────────────────────────────────────────┤
│  ## Workspace                             │  工作区路径（始终）
│  ## Documentation              [full only]  │  文档路径
│  ## Sandbox                    [if enabled]  │  沙箱信息
│  ## User Identity              [full only]  │  用户身份
│  ## Current Date & Time        [if timezone] │  日期时间
├──────────────────────────────────────────┤
│  ## Workspace Files (injected)            │  注入文件说明
│  ## Reply Tags                 [full only]  │  回复标签
│  ## Messaging                  [full only]  │  消息指引
│  ## Voice (TTS)                [full + tts]  │  语音提示
├──────────────────────────────────────────┤
│  ## Group Chat Context / Subagent Context  │  额外上下文
│  ## Reactions                  [if channel]  │  表情反应指导
│  ## Reasoning Format           [if tag hint] │  推理格式
├──────────────────────────────────────────┤
│  # Project Context                        │  引导文件内容
│    ## AGENTS.md                           │
│    ## SOUL.md                             │
│    ## TOOLS.md                            │
│    ## ...                                 │
├──────────────────────────────────────────┤
│  ## Silent Replies             [full only]  │  静默回复
│  ## Heartbeats                 [full only]  │  心跳机制
│  ## Runtime                               │  运行时元数据
└──────────────────────────────────────────┘
```

最终，所有行通过 `lines.filter(Boolean).join("\n")` 拼接为一个完整的字符串。一个典型的 full 模式提示词可能有 **3,000-10,000 字符**（不含注入的上下文文件），加上 AGENTS.md 等文件后可以达到 **20,000-50,000 字符**。

17.1.2 提示词参数（src/agents/system-prompt-params.ts）

提示词模板中引用了许多运行时信息——用户时区、当前时间、Git 仓库根目录等。这些参数由 `buildSystemPromptParams` 函数统一解析。

SystemPromptRuntimeParams 类型

```
// src/agents/system-prompt-params.ts
export type RuntimeInfoInput = {
  agentId?: string;        // Agent 标识
  host: string;            // 主机名
  os: string;              // 操作系统
  arch: string;            // CPU 架构
  node: string;            // Node.js 版本
  model: string;           // 当前模型
  defaultModel?: string;   // 默认模型
  channel?: string;        // 消息通道
  capabilities?: string[];// 通道能力
  channelActions?: string[]; // 通道支持的操作
  repoRoot?: string;       // Git 仓库根目录
};

export type SystemPromptRuntimeParams = {
  runtimeInfo: RuntimeInfoInput;
  userTimezone: string;
  userTime?: string;
  userTimeFormat?: ResolvedTimeFormat;
};
```

参数解析流程

```
export function buildSystemPromptParams(params: {
  config?: OpenClawConfig;
  agentId?: string;
  runtime: Omit<RuntimeInfoInput, "agentId">;
  workspaceDir?: string;
  cwd?: string;
}): SystemPromptRuntimeParams {
  // 1. 解析 Git 仓库根目录
  const repoRoot = resolveRepoRoot({
    config: params.config,
    workspaceDir: params.workspaceDir,
    cwd: params.cwd,
  });
  // 2. 解析用户时区
  const userTimezone = resolveUserTimezone(
    params.config?.agents?.defaults?.userTimezone
  );
  // 3. 解析时间格式偏好（12/24 小时制）
  const userTimeFormat = resolveUserTimeFormat(
    params.config?.agents?.defaults?.timeFormat
  );
  // 4. 格式化当前时间
  const userTime = formatUserTime(new Date(), userTimezone, userTimeFormat);

  return {
    runtimeInfo: { agentId: params.agentId, ...params.runtime, repoRoot },
    userTimezone,
    userTime,
    userTimeFormat,
  };
}
```

Git 仓库根目录检测

`resolveRepoRoot` 函数负责找到当前项目的 Git 根目录。它采用三级优先策略：

```typescript
function resolveRepoRoot(params: {
  config?: OpenClawConfig;
  workspaceDir?: string;
  cwd?: string;
}): string | undefined {
  // 优先级 1：配置文件显式指定
  const configured = params.config?.agents?.defaults?.repoRoot?.trim();
  if (configured) {
    const resolved = path.resolve(configured);
    if (fs.statSync(resolved).isDirectory()) return resolved;
  }
  // 优先级 2：从工作区目录或 CWD 向上查找 .git
  const candidates = [params.workspaceDir, params.cwd].filter(Boolean);
  for (const candidate of candidates) {
    const root = findGitRoot(path.resolve(candidate));
    if (root) return root;
  }
  return undefined;
}
```

`findGitRoot` 的实现使用了经典的"向上遍历"算法——从起始目录开始，逐级检查父目录中是否存在 `.git` 文件或目录，最多向上搜索 12 级：

```typescript
function findGitRoot(startDir: string): string | null {
  let current = path.resolve(startDir);
  for (let i = 0; i < 12; i += 1) {
    const gitPath = path.join(current, ".git");
    try {
      const stat = fs.statSync(gitPath);
      if (stat.isDirectory() || stat.isFile()) return current;
    } catch { /* 继续向上 */ }
    const parent = path.dirname(current);
    if (parent === current) break; // 已到达根目录
    current = parent;
  }
  return null;
}
```

> **衍生解释——`.git` 可以是文件？**

> 通常 `.git` 是一个目录，包含 Git 仓库的所有元数据（objects、refs 等）。但在 Git submodule 和 `git worktree` 场景下，`.git` 会是一个**文件**，内容指向实际的 `.git` 目录。因此 `findGitRoot` 同时检查了 `isDirectory()` 和 `isFile()` 两种情况。

时区与时间格式

时区解析（`resolveUserTimezone`）优先使用配置值，回退到系统的 `Intl.DateTimeFormat().resolvedOptions().timeZone`：

```
// src/agents/date-time.ts
export function resolveUserTimezone(configured?: string): string {
  const trimmed = configured?.trim();
  if (trimmed) {
    try {
      // 验证时区有效性
      new Intl.DateTimeFormat("en-US", { timeZone: trimmed }).format(new Date());
      return trimmed;
    } catch { /* 忽略无效时区 */ }
  }
  return Intl.DateTimeFormat().resolvedOptions().timeZone ?? "UTC";
}
```

时间格式检测（`resolveUserTimeFormat`）则会根据操作系统进行不同的检测：

- **macOS**：读取 `defaults read -g AppleICUForce24HourTime`
- **Windows**：通过 PowerShell 读取 `(Get-Culture).DateTimeFormat.ShortTimePattern`
- **其他**：使用 `Intl.DateTimeFormat` 格式化 13:00 查看结果是否包含 "13"

最终的时间格式化结果类似："Wednesday, February 18th, 2026 — 8:49 AM"。

17.1.3 提示词报告（`src/agents/system-prompt-report.ts`）

系统提示词构建完成后，OpenClaw 会生成一份**诊断报告**（`SessionSystemPromptReport`），用于监控和调试提示词的 token 消耗。这对于防止提示词膨胀导致上下文窗口溢出至关重要。

报告结构

```typescript
// src/agents/system-prompt-report.ts（推断的报告结构）
type SessionSystemPromptReport = {
  source: string;          // 提示词来源标识
  generatedAt: number;     // 生成时间戳
  sessionId?: string;
  sessionKey?: string;
  provider?: string;
  model?: string;
  workspaceDir?: string;
  bootstrapMaxChars: number; // 引导文件最大字符数
  sandbox?: { ... };       // 沙箱配置
  systemPrompt: {
    chars: number;           // 总字符数
    projectContextChars: number;   // 项目上下文占用字符
    nonProjectContextChars: number; // 非项目上下文字符
  };
  injectedWorkspaceFiles: Array<{
    name: string;
    path: string;
    missing: boolean;
    rawChars: number;        // 原始字符数
    injectedChars: number;   // 注入后字符数
    truncated: boolean;      // 是否被截断
  }>;
  skills: {
    promptChars: number;     // 技能提示总字符数
    entries: Array<{ name: string; blockChars: number }>;
  };
  tools: {
    listChars: number;       // 工具列表字符数
    schemaChars: number;     // 工具 JSON Schema 总字符数
    entries: Array<{
      name: string;
      summaryChars: number;
      schemaChars: number;
      propertiesCount: number | null;
    }>;
  };
};
```

技能解析：parseSkillBlocks

报告需要知道每个技能占用了多少字符。 `parseSkillBlocks` 使用正则表达式从技能 XML 中提取各个 `<skill>` 块：

```
function parseSkillBlocks(
  skillsPrompt: string
): Array<{ name: string; blockChars: number }> {
  const blocks = Array.from(
    skillsPrompt.matchAll(/<skill>[\s\S]*?<\/skill>/gi)
  ).map((match) => match[0] ?? "");

  return blocks.map((block) => {
    const name = block.match(/<name>\s*([^<]+?)\s*<\/name>/i)?.[1]?.trim()
      || "(unknown)";
    return { name, blockChars: block.length };
  }).filter((b) => b.blockChars > 0);
}
```

工具统计：buildToolsEntries

每个工具的描述长度和 JSON Schema 大小都会被统计：

```
function buildToolsEntries(
  tools: AgentTool[]
): Array<{ name; summaryChars; schemaChars; propertiesCount }> {
  return tools.map((tool) => {
    const summaryChars = (tool.description || tool.label || "").length;
    const schemaChars = tool.parameters
      ? JSON.stringify(tool.parameters).length
      : 0;
    const propertiesCount = tool.parameters?.properties
      ? Object.keys(tool.parameters.properties).length
      : null;
    return { name: tool.name, summaryChars, schemaChars, propertiesCount };
  });
}
```

注入文件统计：buildInjectedWorkspaceFiles

对于每个引导文件，报告记录其原始大小、注入后的大小、以及是否被截断：

```
function buildInjectedWorkspaceFiles(params: {
  bootstrapFiles: WorkspaceBootstrapFile[];
  injectedFiles: EmbeddedContextFile[];
  bootstrapMaxChars: number;
}): SessionSystemPromptReport["injectedWorkspaceFiles"] {
  const injectedByName = new Map(
    params.injectedFiles.map((f) => [f.path, f.content])
  );
  return params.bootstrapFiles.map((file) => {
    const rawChars = file.missing ? 0 : (file.content ?? "").trimEnd().length;
    const injected = injectedByName.get(file.name);
    const injectedChars = injected ? injected.length : 0;
    const truncated = !file.missing && rawChars > params.bootstrapMaxChars;
    return {
      name: file.name,
      path: file.path,
      missing: file.missing,
      rawChars,
      injectedChars,
      truncated,
    };
  });
}
```

报告的主要用途

1. **Token 预算监控**：通过 `systemPrompt.chars` 和各子项的字符数，运维人员可以快速发现哪个部分导致了提示词膨胀。
2. **截断告警**：当引导文件超过 `bootstrapMaxChars` （默认 20,000 字符）被截断时，报告中的 `truncated` 标志会触发告警。
3. **工具审计**： `tools.schemaChars` 统计了所有工具 JSON Schema 的总大小。当 MCP 提供的外部工具过多时，Schema 可能消耗大量上下文空间，报告帮助识别这类问题。
4. **技能开销分析**： `skills.entries` 列出每个技能块的字符数，帮助识别体积过大的技能定义。

本节小结

1. **系统提示词**是 OpenClaw 控制 Agent 行为的核心机制，采用**模块化组装**模式——十余个功能段落根据运行时条件动态拼接。

2. **三种 PromptMode**（full/minimal/none）为不同场景提供不同详尽度的提示词：主 Agent 获得完整指令，子 Agent 获得精简版本，极简模式仅保留身份声明。
3. **运行时参数**（`buildSystemPromptParams`）负责解析时区、时间格式、Git 仓库根目录等环境信息，采用"配置优先、系统检测回退"的策略。
4. **提示词报告**（`buildSystemPromptReport`）为每次提示词生成创建详细的统计信息，是防止上下文窗口溢出的第一道防线。
5. 提示词的**体积管理**是一个关键挑战——引导文件截断、工具 Schema 统计、技能字符数监控共同构成了 OpenClaw 的 token 预算管理体系。